INFORME PRELIMINAR – DESAFÍO 2

Amigos para siempre, con Ude@Book

Curso: Informática II

Semestre: 2026-2

Alejandro Macias Rodríguez

CONTEXTUALIZACION:

El desafío consiste en desarrollar un programa que simule las funcionalidades básicas de una red social denominada Ude@Book, utilizando el paradigma de programación orientada a objetos.

La plataforma debe permitir la administración de usuarios, amistades (solicitudes y amistades ya establecidas), álbumes de fotografías y mensajes. También debe ofrecer funcionalidades como el inicio de sesión, la visualización de fotografías de amigos (solo las rutas), las sugerencias de amistad y la consulta de mensajes.

Debemos también tener presente la eficiencia en el uso de los recursos computacionales, por lo que se requiere implementar estructuras de datos propias mediante memoria dinámica y medir tanto las iteraciones ejecutadas como el consumo de memoria de las funcionalidades.

Ya como consideraciones finales del contexto de este problema, debemos tener presente las prohibiciones de: herencia, struct y STL.

ANALISIS DEL PROBLEMA:

Es necesario identificar los elementos que conforman la plataforma, las relaciones existentes entre ellos y las operaciones que se deben realizar sobre la información, a continuación, explico lo entendido acerca de las funcionalidades:

Gestión de usuarios y almacenamiento: Cada usuario debe contar con un nombre de usuario único y sus respectivos datos personales. El sistema debe permitir recuperar la información desde archivos (almacenamiento permanente) y mantener actualizados los datos que sean modificados durante la ejecución (almacenamiento temporal durante el tiempo de ejecución). También debe garantizar que las relaciones entre usuarios, mensajes, álbumes y fotografías se conserven correctamente.

Relaciones de amistad: Las amistades deben ser mutuas, de manera que cuando dos usuarios establezcan una amistad, ambos queden relacionados. Además, se debe evitar el envío de solicitudes a usuarios que ya sean amigos o con quienes exista una solicitud pendiente que impida realizar la operación.

Visualización de amigos: Al iniciar sesión, se deben seleccionar aleatoriamente tres fotografías de perfil de amigos para ser mostradas. Se debe evitar repetir fotografías dentro de una misma selección y controlar que la actualización del grupo de fotografías solamente pueda realizarse una vez.

Administración de álbumes y fotografías: Cada usuario puede poseer varios álbumes con diferentes niveles de privacidad. Por lo tanto, se debe controlar cuáles álbumes pueden ser

consultados por otros usuarios. Las fotografías deben poder recorrerse circularmente y permitir aumentar o disminuir su cantidad de likes mediante las operaciones correspondientes.

Sugerencias de amistad: El sistema debe identificar usuarios que sean amigos de los amigos del usuario actual, sin incluir sus amistades existentes. Posteriormente, debe calcular la cantidad de amigos en común y ordenar los candidatos para presentar las sugerencias solicitadas. Para esto tenemos la sugerencia de la matriz de adyacencia, sin embargo, tenemos la libertad de decidir.

Gestión de mensajes: Los usuarios deben poder intercambiar mensajes con sus amigos, consultar los mensajes recibidos que todavía no han sido leídos y realizar búsquedas dentro de un intervalo de fechas. Para ello, se debe conservar la información del emisor, destinatario, contenido, fecha de envío y estado de lectura.

Medición de recursos: Cada funcionalidad debe permitir conocer la cantidad de iteraciones necesarias para ejecutarse, incluyendo las realizadas indirectamente. También se debe contabilizar la memoria utilizada por las estructuras de datos, objetos y variables.

DISEÑO:

Hasta este punto del informe final, mantuve la información del informe preliminar correspondiente a las secciones de contextualización y análisis del problema, sin embargo, de aquí en adelante se encuentra la información más enfocada al informe preliminar.

Durante el desarrollo de mi programa mantuve el diseño planteado en el informe preliminar, sin embargo, de acuerdo a mis necesidades hubo unos pequeños cambios en la implementación final.

En el programa final encontraremos en total las 6 clases planteadas desde el principio:

- Usuario: representa a cada persona registrada en la red social. Almacena sus datos personales, credenciales y álbumes de fotografías.

- Plataforma: es la clase principal encargada de coordinar el funcionamiento de Ude@Book. Administra los usuarios, las amistades, las solicitudes y los mensajes, además de gestionar el almacenamiento permanente.

- Solicitud amistad: representa una solicitud enviada entre dos usuarios. Permite identificar al emisor y al destinatario para gestionar las solicitudes pendientes.

- Mensaje: representa la información intercambiada entre usuarios. Contiene el emisor, destinatario, texto, fecha de envío y estado de lectura.

- Foto: representa una fotografía con su ruta, fecha de captura, tamaño y cantidad de likes. Incluye sobrecarga de operadores para aumentar y disminuir los likes.

- Álbum: representa una colección de fotografías perteneciente a un usuario. Administra las fotos mediante memoria dinámica y establece su nivel de privacidad.

Todas creadas en ese orden. Pero al final surgió una nueva clase llamada medidorrecursos, clase auxiliar encargada de contabilizar las iteraciones realizadas durante las operaciones y apoyar el cálculo del consumo estimado de memoria.

Ahora tenemos que hablar de la base de datos, esta concuerda con la propuesta inicialmente, a excepción de un detalle y es que el archivo fotos.txt no fue necesario implementarlo, ya que, la información de las fotos quedo integrada en álbumes.txt, sin necesitar un archivo propio.

- usuarios.txt: almacena la información personal y las credenciales de los usuarios registrados, permitiendo reconstruir sus objetos al iniciar el programa.

- amistades.txt: conserva las relaciones de amistad entre usuarios, utilizadas para reconstruir la matriz de adyacencia en memoria RAM (esta matriz es dinámica, esto precisamente fue discutido en la ultima clase que tuvimos relacionada con el desafío 2).

- solicitudes.txt: almacena las solicitudes de amistad pendientes, indicando quién las envió y quién debe responderlas.

- mensajes.txt: guarda los mensajes intercambiados entre usuarios, incluyendo emisor, destinatario, contenido, fecha de envío y estado de lectura.

- albumes.txt: almacena la información de los álbumes y sus fotografías, incluyendo nombres, rutas, niveles de privacidad, fechas, tamaños y likes.

Estos archivos permiten conservar la información después de cerrar el programa. Durante la ejecución, sus datos se cargan en objetos y estructuras dinámicas en la memoria RAM, donde realizamos las operaciones correspondientes.

ALGORITMOS ESENCIALES:

A continuación, voy a describir en alto nivel como indica el enunciado los algoritmos esenciales para el funcionamiento de mi programa.

1. Administración de memoria dinámica

Utilizamos arreglos dinámicos para almacenar usuarios, solicitudes, mensajes, álbumes y fotografías. Para esto implemente ciertas validaciones para no tener problemas con la capacidad de los arreglos. Esto permite aumentar la cantidad de elementos durante la ejecución sin conocerla anticipadamente.

2. Gestión de amistades mediante matriz de adyacencia

Representamos las amistades mediante una matriz dinámica de valores booleanos, almacenada en un arreglo unidimensional (al ser unidimensional tenemos que hacer operaciones muy parecidas al anterior desafío, donde también almacenábamos un tablero en un arreglo unidimensional). Cuando dos usuarios establecen una amistad, marcamos las dos posiciones correspondientes para garantizar que la relación sea mutua. Al ampliar la capacidad de usuarios, también reorganizamos la matriz conservando las amistades existentes.

3. Cálculo de sugerencias de amistad

Recorremos las relaciones de amistad para identificar posibles candidatos entre los amigos de nuestros amigos. Excluimos los usuarios que ya son amigos y aquellos con solicitudes pendientes. Posteriormente, calculamos cuántos amigos en común existen con cada candidato y organizamos los resultados de mayor a menor mediante un ordenamiento manual.

4. Selección aleatoria ponderada de fotografías

A cada amigo le asignamos un peso según la cantidad de amigos en común: 1, 2,5 o 3,5. Para facilitar los cálculos utilizamos los pesos equivalentes 2, 5 y 7. Realizamos sorteos aleatorios considerando esas prioridades y evitamos repetir usuarios dentro del grupo seleccionado. La actualización solo se permite una vez por sesión.

5. Administración y búsqueda de mensajes

Los mensajes se insertan manualmente en orden cronológico según su fecha de envío, desplazando los elementos necesarios dentro del arreglo dinámico. Para buscar mensajes recibidos dentro de un intervalo de fechas, recorremos la estructura y comprobamos cuáles pertenecen al destinatario y cumplen las fechas solicitadas. También administramos su estado de leído o no leído.

6. Recorrido circular de fotografías

Cada álbum administra sus fotografías mediante un arreglo dinámico. Para realizar el recorrido circular, controlamos la posición actual: cuando avanzamos desde la última fotografía regresamos a la primera, y cuando retrocedemos desde la primera pasamos a la última. También permitimos modificar los likes mediante los operadores sobrecargados `++` y `--`.

7. Almacenamiento permanente y recuperación de información

Al iniciar el programa leemos los cinco archivos de almacenamiento y reconstruimos los objetos y estructuras dinámicas en RAM. Las operaciones se realizan sobre estas

estructuras y, cuando corresponde, guardamos los cambios nuevamente en los archivos. De esta manera, la información se conserva entre ejecuciones.

8. Medición del consumo de recursos

Implementamos un contador que registra las iteraciones realizadas por nuestros algoritmos, incluyendo las operaciones internas llamadas por otras funciones. Adicionalmente, calculamos una estimación de memoria considerando los objetos y las capacidades reservadas de las estructuras dinámicas, junto con los elementos temporales contemplados por nuestro medidor.

PROBLEMAS AFRONTADOS:

Realmente durante el desarrollo de este programa tuve una infinidad de problemas, sin embargo, aquí voy a describir problemas enteramente relacionados con las temáticas importantes del curso, espero los demás problemas conversarlos en la sustentación.

- Problemas relacionados con la administración de memoria dinámica:

Una de las primeras dificultades fue comprender y administrar los punteros, especialmente usuario**, y el crecimiento de los arreglos dinámicos. Para resolverlo, implementamos procedimientos que reservan mayor capacidad, trasladan los elementos y liberan la memoria anterior. También tuvimos que asegurar que las amistades se conservaran cuando aumentaba el tamaño de la matriz de adyacencia.

- Problemas con los archivos permanentes:

  En un principio empecé de lleno con el desarrollo modelando esta situación con clases y objetos. El problema fue que al principio estaba desarrollando sin pensar tanto en la relación de mis clases con los archivos permanentes, y esto fue un choque muy fuerte. Me tocó parar el desarrollo y pensar muy bien cómo iban a ser mis archivos permanentes y cómo se relacionan con los objetos que estaba creando en tiempo de ejecución, luego de haber planteado muy bien cómo iban a ser mis archivos, fue maravilloso seguir desarrollando.

- Problemas con copias de objetos:

  Otro aspecto que requirió especial cuidado fue la copia de objetos que contienen punteros. Una copia superficial podría provocar que dos objetos compartieran la misma memoria y ocasionar problemas al liberarla. Por eso implementamos constructores de copia.

- Generación base de datos con mil usuarios:

  Finalmente me gustaría mencionar el problema que tuve generando la base de datos exigida en el enunciado del desafío, nada más de pensar en esto me hace creer que este

fue el problema más estresante del desafío, sin embargo, luego recuerdo todos los demás problemas y concluyo que este fue uno más del montón. Logré crear la base de datos de manera estandarizada y lo más importante de todo, cumpliendo los requisitos para la demostración del funcionamiento.

EVOLUCION DE LA SOLUCIÓN:

Inicialmente planteé una solución basada en seis clases principales, encargadas de representar los elementos de la red social. Durante la implementación decidí conservar estas clases, pero fue necesario incorporar una clase auxiliar llamada medidorrecursos para facilitar el conteo de iteraciones y la estimación de memoria.

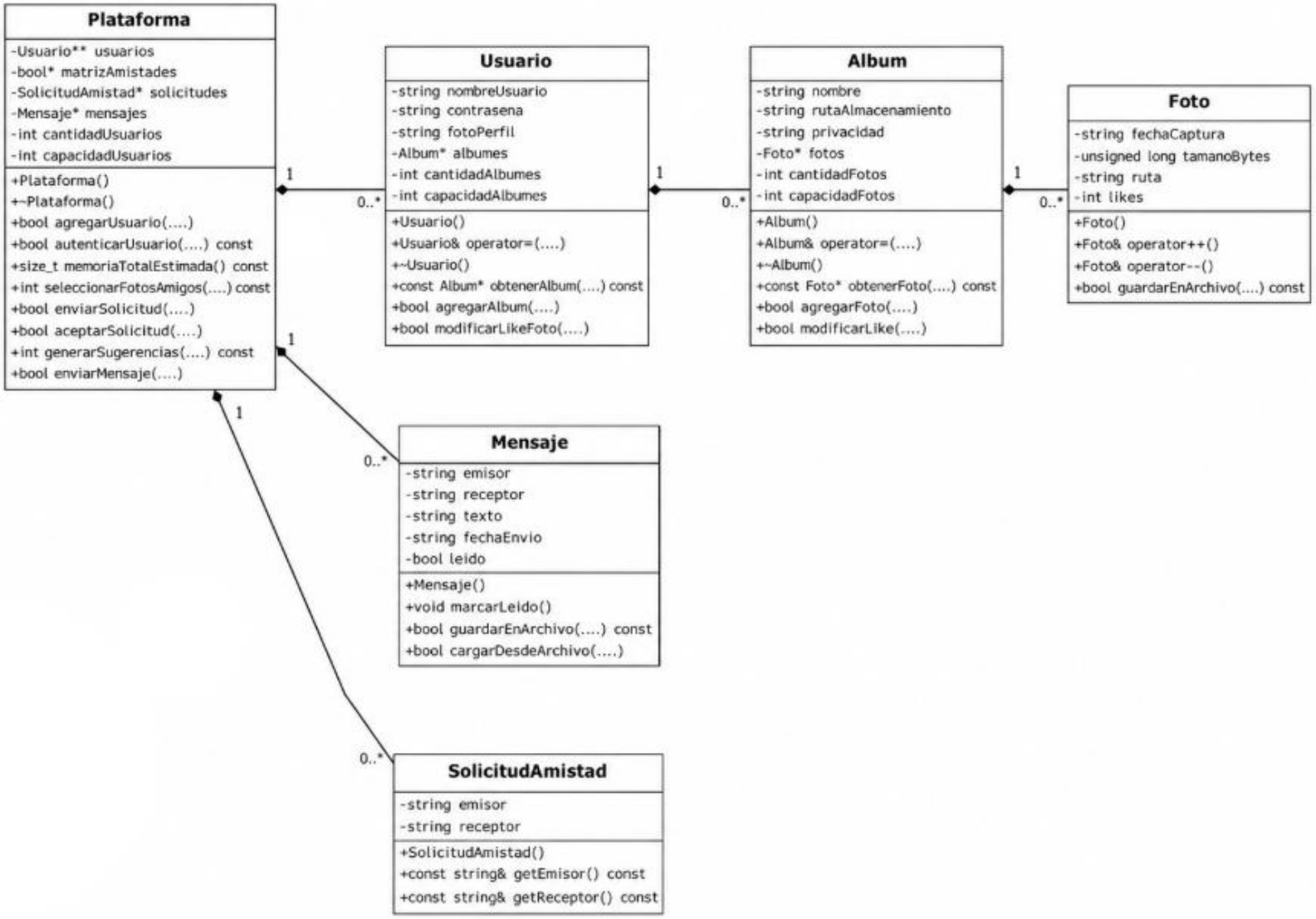
El desarrollo comenzó con la clase usuario y posteriormente plataforma, donde implementé los primeros arreglos dinámicos. Después incorporé el almacenamiento permanente, las relaciones de amistad mediante una matriz de adyacencia, las solicitudes, los mensajes y las sugerencias de amistad. Finalmente, implementé los álbumes, fotografías, autenticación y selección aleatoria ponderada.

Uno de los cambios respecto al diseño preliminar fue la organización de los archivos permanentes. Inicialmente propuse seis archivos, pero finalmente utilicé cinco, almacenando la información de las fotografías dentro de albumes.txt.

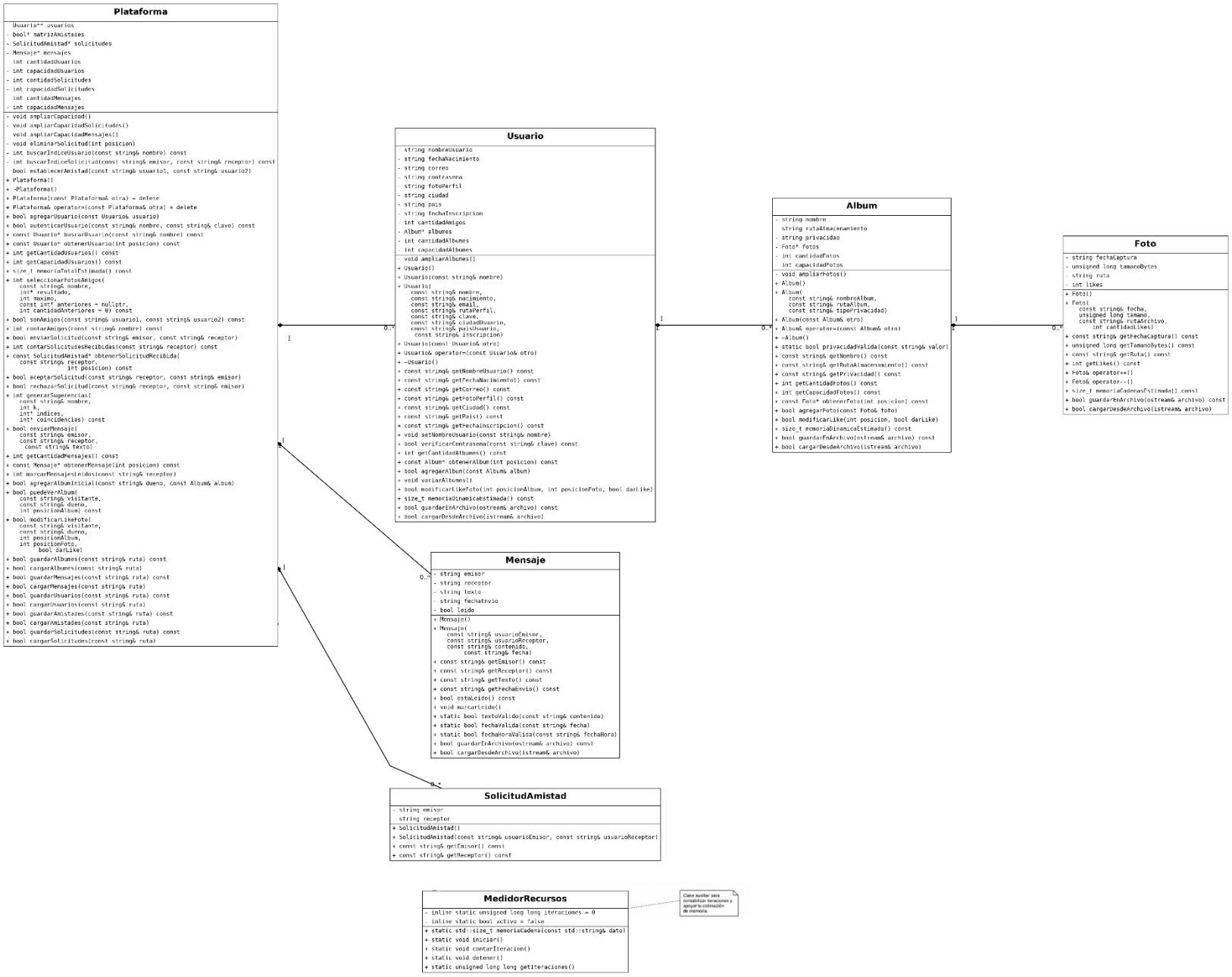
A medida que avanzó la implementación también fue necesario incorporar constructores de copia, sobrecarga de operadores y mecanismos de liberación de memoria. Finalmente, realicé pruebas con 1000 usuarios registrados, comprobando las funcionalidades principales y sus respectivas métricas de recursos.

DIAGRAMAS DE CLASE:

A continuación, tenemos los diagramas de clases tanto simplificado como completo, esta vez si son fieles a mi código:



Esta sería la versión simplificada del diagrama de clases, quedo muy conforme con el resultado de este, sin embargo, solo quede con una duda que la verdad en medio de todo el caos del desarrollo pase por alto discutir bien sea con el profe de teoría o con el de laboratorio, y es la siguiente, yo decidí que la estructura de mi programa fuera de esta manera, más específicamente que usuario, mensaje y solicitudamistad fueran parte de plataforma, pero la duda es que en el informe preliminar, incluí dos relaciones de asociación simple entre solicitudamistad y usuario y también entre mensaje y usuario, pero tratando de ser mas formal en la entrega y también basándome en los ejemplos vistos en clase decidí dejarlo así para esta entrega final pero como mínimo dejar evidenciada esta pequeña duda. Además adjunto también el diagrama completo.



es difícil poder evidenciar visualmente este diagrama debido a su tamaño, pero en caso de ser necesario podríamos revisar el archivo original en la sustentación.

CONCLUSIONES:

Con el desarrollo de Ude@Book entendí una vez mas la importancia del proceso de planear y diseñar la solución antes de empezar a codificar, con el anterior desafío pensé que de verdad había entendido la importancia de ese proceso, pero con este desafío me doy cuenta que realmente no es hasta tener un problema de este estilo que uno de verdad reconoce que, si no realiza un buen análisis y diseño, hay atas probabilidades de fallar en el intento.

Otro aprendizaje importante fue comprender la diferencia entre la información almacenada permanentemente en archivos y los objetos que existen en la memoria RAM durante la ejecución. También pude entender mejor la utilidad de estructuras como la matriz de

adyacencia para representar las amistades y facilitar operaciones como las sugerencias de usuarios.

También me gustaría mencionar una de las cosas que más aprendí desarrollando este programa y es la importancia de uno de los pilares fundamentales de la programación orientada a objetos llamado encapsulamiento, ya con el ejemplo que vimos en clase del objeto marcador uno podría empezar a tener nociones de ese concepto, pero con el desarrollo de este desafío consolida por completo ese conocimiento.

Para cerrar las conclusiones y dejando de lado las temáticas del curso me gustaría expresar la cantidad de sentimientos encontrados durante este desafío, una vez más pensé que ni siquiera iba a llegar con algo concreto a la sustentación, este pensamiento estuvo mas presente en este desafío que en el anterior, sin embargo en todo momento intente mas que solucionar la totalidad del problema, demostrarme a mi mismo que independientemente de cualquier factor externo si me lo proponía de verdad lo iba a lograr.

ANEXO:

Para finalizar este informe, dejo nuevamente anexada la imagen de mi cuaderno donde lo que mas rescato una vez mas es la siguiente frase: “Es importante distinguir en todo momento lo almacenado de manera permanente de lo que esta almacenado en el programa durante la ejecución”

Desafio 2.

nos piden desarrollar una especie de facebook en la consola que permita administrar: usuarios, amistades fotografias, albumes y mensajes

Notas importantes:

- distinguir en todo momento lo almacenado de manera permanente de lo que esta almacenddo en el programa durante la ejecucion
- diferencia entre clase y objeto: "molde de galletas" "una galleta en particular"
- Consultar sobre la matriz de adyacencia

